

Utilización de la (DFS)

- Versión inicial propia (pegar el código, aunque tuviera errores).

```
def depthFirstSearch(problem: SearchProblem):
    frontier = utils.Stack()
    startState = problem.getStartState()
    frontier.push((startState, []))
    visited = set()
    visited.add(startState)
    while not frontier.isEmpty():
        state, actions = frontier.pop()
        if problem.isGoalState(state):
            return actions
        for successor, action, stepCost in problem.getSuccessors(state):
            if successor not in visited:
                visited.add(successor)
                frontier.push((successor, actions + [action]))
    return []
```

- Prompts utilizados:

Quería saber cuál era la mejor variación del algoritmo que me podría servir, por lo que solicite ayuda conceptual y de búsqueda para encontrar documentación que explicara recorridos DFS en grillas:

"Necesito realizar un recorrido DFS sobre una grilla para encontrar una solución al problema de búsqueda, ¿de qué forma puedo evitar ciclos y hacerlo de una manera familiar? He trabajado antes con recorridos en árboles y grafos, pero no sé si es mejor realizar recorrido en grafo de estados o en árbol de decisión."

Posteriormente, después de solicitar verificación de la coherencia del código inicial recibí la siguiente sugerencia:

"Aquí cada elemento de la pila guarda solo una acción individual (action), no el historial acumulado como debería ser. Luego, cada vez que haces pop, agregas esa acción a una lista compartida actions con .append(). El problema: en DFS con pila, cuando el algoritmo "retrocede" (backtracking) — es decir, cuando agota un camino sin éxito y saca de la pila un estado que pertenece a una rama completamente distinta — tu lista actions no se limpia ni se corrige. Sigue creciendo con acciones de ramas abandonadas que ya no llevan a ningún lado"

- Sugerencias relevantes recibidas.

Para la metodología de la construcción del recorrido recibí la siguiente sugerencia:

"La grilla se maneja como un grafo, no un árbol. En un árbol, cada nodo tiene un único padre y no hay forma de "volver" a un ancestro por otro camino. En una grilla, cada celda tiene hasta 4 vecinos, y esos vecinos pueden llevarte de vuelta a una celda que ya visitaste (por ejemplo: derecha, abajo, izquierda, arriba te devuelve al punto de partida). Eso es literalmente un ciclo. Por eso los recorridos de árbol que ya conoces (donde nunca te preocupas por "no volver atrás"

- Cambios aceptados, rechazados o adaptados, con la razón.

Dentro de las ideas de documentación que Claude ofreció, partí por las que especificaba un recorrido no recursivo orientado a graph search, y descarté cualquier otra opción. Empecé a realizar la función DFS que fuera funcional para el contexto del taller. Utilicé las documentaciones sugeridas por Claude para saber el orden que debe seguir el algoritmo y también me basé en la función de búsqueda de grafo creada por mí en el curso de Estructuras de datos y Algoritmos. Al intentar probarlo, no funcionó y recurrí después de varios intentos fallidos, recurrí de nuevo a Claude para lograr entender que parte del algoritmo estaba realizando erróneamente. Claude fue capaz de explicarme que parte del pseudocódigo de las documentaciones no estaba entendiendo

- Cómo se verificó que funciona.

Una vez realizados los cambios, probé nuevamente la función con los casos de búsqueda que ofrece la interfaz. La verificación se realizó comprobando que el robot pudiera encontrar un camino hasta su meta en cada mapa y que las acciones retornadas correspondiera únicamente al camino tomado. Adicionalmente me comprobó que el algoritmo de búsqueda no permite al robot quedarse atrapado en ciclos.

Reflexión final:

Este ejercicio me permitió confirmar que el planteamiento teórico de un algoritmo puede tener distintos modos de implementación, los cuales varían dependiendo del contexto y el ambiente en el cual el agente se desarrolle. Gracias a la explicación dada por la IA, entendí que la solución a los ciclos infinitos no estaba en la lógica del código, sino en la estructura que utilizara para la búsqueda. Adicionalmente, pude comprender errores que cometí dentro de la construcción de la función al guardar las acciones individualmente en vez del camino completo, lo que me permitió entender mejor el algoritmo de búsqueda y posteriormente corregirlo para que fuera funcional. Los prompts utilizados se encuentran en el anexo B.